



ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

COURSE CODE: EEE 4308

COURSE NAME: DIGITAL ELECTRONICS LAB

PROJECT: SAP-1 Architecture based 8-bit Computer Design and Implementation.

Phase: C

GROUP NAME: Boolean Squad

GROUP MEMBERS:

1. IKRAM HOSSAIN (230021111)
2. NADMAN IBNE MAMUN (230021114)
3. FARHAN ISHRAQ (230021124)
4. SAMIN YASAR SAMI (230021138)
5. ZAVID ATAHAR SAMIN (230021155)
6. AGHA MAKINUN AMIN CHOWDHURY (230021232)
7. RAZEEN ABRAR RAHMAN (230021215)

OBJECTIVES:

The objective of **phase C** is to design and implement the following parts:

1. Arithmetic and Logic Unit (ALU)
2. Instruction Register
3. Controller Sequencer
4. MAR & Address Selector

DESCRIPTION:

Arithmetic and Logic Unit (ALU):

The **Arithmetic Logic Unit (ALU)** serves as the primary calculation engine of the system, designed to perform fundamental arithmetic operations on 8-bit data. Its core functionality revolves around an 8-bit **Adder-Subtractor** circuit that utilizes **2's complement logic** to handle both addition and subtraction seamlessly. By using **XOR gates** as programmable inverters, the ALU can either pass data from the B-Register directly for addition or flip the bits to create a 1's complement for subtraction. When the **SU (Subtract)** signal is active, the circuit adds a "1" to the least significant bit (LSB) via the carry-in (C_0) of the first adder, completing the 2's complement process. The final 8-bit result is then interfaced with the system bus through a **74HCT245 tri-state buffer**, ensuring that the result is only driven onto the bus when the **EO (Enable Out)** signal is triggered by the controller.

Instruction Register:

The **Instruction Register (IR)** is a critical 8-bit storage unit that facilitates the "Fetch" and "Execute" phases of the computer's operation. It is constructed using two **74LS173** 4-bit D-type registers, which capture data from the system bus when the **II (Instruction In)** signal is activated during the fetch cycle. Once loaded, the IR splits the 8-bit word into two distinct functional parts: the upper 4 bits (**Opcode**) and the lower 4 bits (**Operand**). The Opcode bits are sent directly to the **Control Unit** to determine the operation to be performed, while the Operand bits (typically a memory address) can be outputted back onto the bus via the **IO (Instruction Out)** signal. This dual-output design allows the IR to act as a bridge between the software instructions stored in memory and the hardware execution units of the CPU.

Controller Sequencer:

The control sequencer acts as the main component that coordinates the movement of data across the common bus translating machine instruction into timed electrical signal. It decides which component sends data to the bus and which one will receive that data for each of the cycle. According to the need it enables or disables different control pins of the several needed components depending. The component depends on the clock pulse from Clock Pulse Generator and takes 4bit input from the higher nibble of the Instruction Register which will direct it to some specific command to be executed.

MAR & Address Selector:

In SAP1 computer, the MAR and Address Selector work together to decide depending on the cycle of operation which source of address should be feed into the RAM. We can say that they are general components used for managing memory operation allowing the processor to access specific data location.

- MAR (Memory Address Register): A 4bit register that stores the memory address currently being accessed.
Function: During Fetch cycle, it gets address from Program Counter via bus and during Execution cycle it gets the address from Instruction Register.
- Address Selector: A quad 2-to-1 MUX that selects which address is passed to RAM.
Mode of Operation:
 1. Program mode: Allows manual selection.
 2. Run mode: Selects the address passed from MAR.

8-Bit Arithmetic Logic Unit (ALU)

The 8-bit ALU is the primary computational engine of the SAP-1 architecture. It is responsible for executing arithmetic instructions by processing data from the **Accumulator (Register A)** and the **B-Register**. The design utilizes 2's complement arithmetic to enable both addition and subtraction using the same core hardware.

A. Addition Strategy

The ALU uses binary full-adder logic. When the **SU (Subtract) control signal** is **LOW (0)**:

- The XOR gates act as "**buffers**," passing the B-Register data to the adders without change.
- The **carry-in (C₀) of the lowest adder** is **0**.
- The system performs: **Result = A + B**.

B. Subtraction Strategy (2's Complement)

When the **SU** signal is **HIGH (1)**, the ALU performs subtraction (by transforming it into $A + (-B)$).

1. **Bitwise Inversion:** XOR gates flip every bit of the B-Register (creating the 1's complement).
2. **Adding the LSB:** By feeding the **SU** signal into the **carry-in (C₀)** of the first adder, we **add 1** to the inverted B-value.
3. The system performs: **Result: $A + (\bar{B} + 1) = A - B$** .

Role of the XOR Gates (Programmable Inversion):

The XOR gate is the "Decision Maker" of the ALU. Its primary role is to control the B-Register's data based on the **SU (Subtract)** signal. In this ALU architecture, the XOR gate acts as a "**Programmable Inverter**." It is the critical component that allows the same adder hardware to perform both addition and subtraction

- **Conditional Inversion (The Add/Subtract Switch):**

By connecting one input of the XOR gate to the **SU (Subtract)** control signal, the gate determines how the ALU processes the B-Register data:

- **Addition (SU = 0):** When the **SU** signal is **LOW**, the XOR gate acts as a buffer. It passes the B-Register data to the adder without any modification.
- **Subtraction (SU = 1):** When the **SU** signal is **HIGH**, the XOR gate inverts every bit of the B-Register (**converting 0s to 1s and 1s to 0s**). This resulting value is known as the **1's Complement**.

- **Implementing 2's Complement Logic:**

To perform binary subtraction, we must use the **2's Complement** method, defined as (**1's Complement + 1**). The XOR gate facilitates this in two ways:

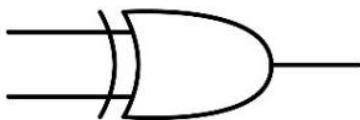
1. It creates the **1's Complement** by inverting the B-Register bits when **SU=1**.
2. The same **SU** signal is fed into the **Carry-In (C₀)** of the first Adder IC. This effectively adds the required "+1" to the inverted bits.

- **Mathematical Result:** $A + (\bar{B} + 1) = A - B$

- **Space and Cost Efficiency:**

Without the use of XOR gates, the circuit would require separate **Inverter ICs** to flip the bits and **Multiplexers** to choose between the original and inverted data. The XOR gate performs both functions, signal passing and signal inversion simultaneously, making the circuit significantly more compact and power-efficient.

XOR Function Table:

				
Gate	Notation	Truth table		
XOR	$A \oplus B$	INPUT	OUTPUT	
		A	B	A XOR B
		0	0	0
		0	1	1
		1	0	1
1	1	0		

Input A (Control - SU)	Input B (Data - B Register)	Output (Y)
0	0	0
0	1	1
1	0	1
1	1	0

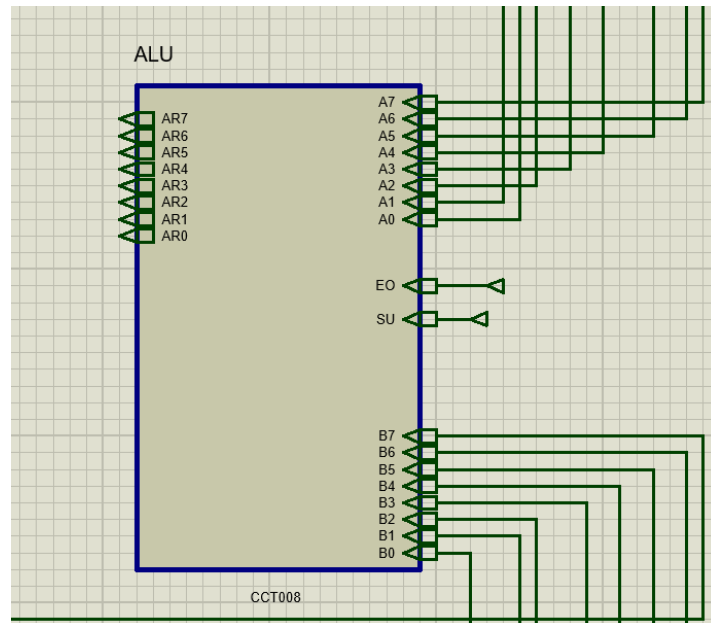
ALU Function Table:

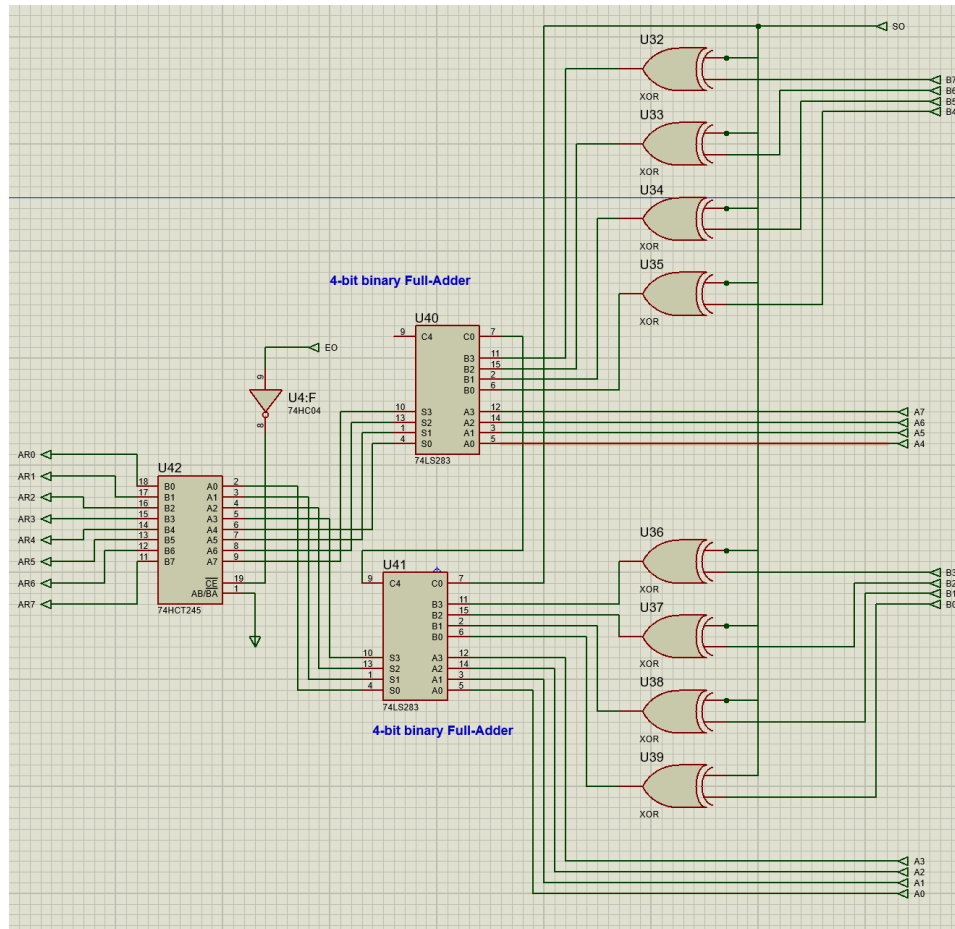
EO (Enable)	SU (Subtract)	ALU State	Result to Bus
0	X	Isolated	High Impedance (Bus Disconnected)
1	0	ADD	$A + B$
1	1	SUB	$A - B$

Algorithm Flow:

1. **Fetch:** Load Register A and Register B from the system bus.
2. **Process:** The Adders continuously calculate the sum/difference based on the **SU** signal.
3. **Output Enable:** When the **EO** signal is triggered, the **74HCT245** activates, pushing the 8-bit result onto the **bus pins (AR₀-AR₇)**

Circuit Diagram:





Component list:

Category	Component ID	Description	Quantity	Purpose
Arithmetic	74LS283	4-Bit Binary Full Adder	2	Core addition logic (cascaded).
Interfacing	74HCT245	Octal Bus Transceiver	1	Tri-state buffer to drive the system bus.
Control	74HC04	Hex Inverter	1	Polarizing control signals (EO).
Visuals	LEDs	Logic Level Indicators	8	Real-time output monitoring.
Gate	XOR	XOR Gate	8	Programmable bitwise inverter for subtraction.

Instruction Register

The **Instruction Register (IR)** in the SAP-1 is an 8-bit, parallel-in/parallel-out register. While it functions similarly to the other registers in the system, its unique output configuration sets it apart.

Here is a breakdown of how it works:

- **Loading Data:** The **Instruction In** signal acts as the control switch. When activated, it allows the 8-bit instruction to flow from the bus into the register. (This is similar to the Shift/Load logic used in the MAR).
- **The Split Output:** Once the instruction is stored, it is divided into two 4-bit sections (nibbles):
 - **The First Nibble (Opcode):** This goes straight to the **Controller-Sequencer** to tell the computer what operation to perform.
 - **The Second Nibble (Address):** This is sent through a **tri-state buffer**. When the **IO (Instruction Out)** signal is triggered, this address is released back onto the bus, where it is typically picked up by the **MAR** for memory access.

IR Operation Algorithm:

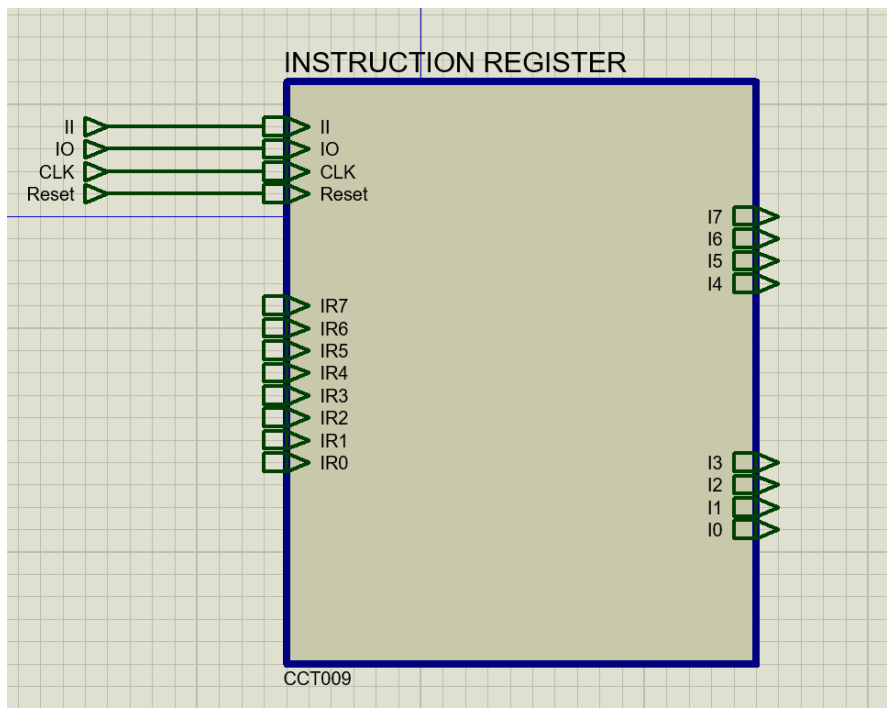
The IR operates based on the control signals provided in your diagram (**II**, **IO**, **CLK**, and **Reset**):

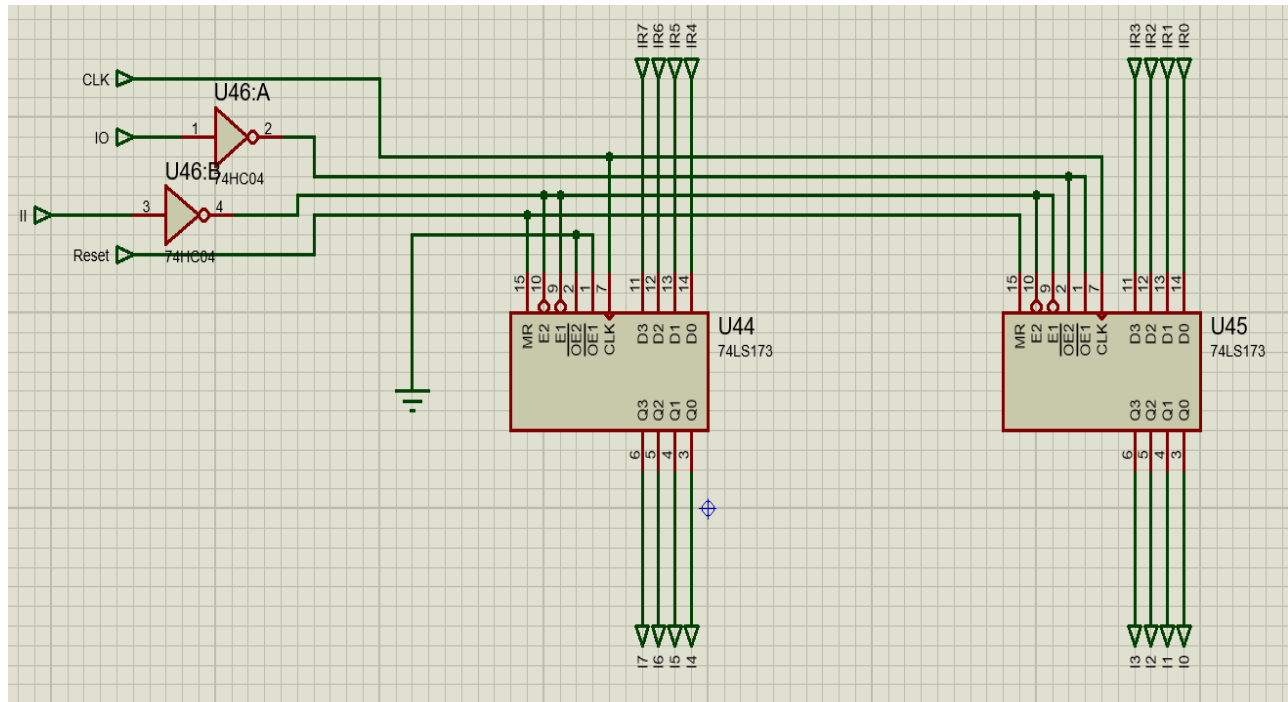
1. **Resting State:** The IR ignores the bus as long as the **II (Instruction In)** signal is inactive.
2. **The Fetch Step:**
 - The Control Unit sets **II = 0** (Active Low for 74LS173).
 - On the next **Rising Edge** of the **CLK**, the 8-bit data currently on the bus is latched into **U44** and **U45**.
3. **Instruction Output:**
 - The **Opcode** bits are permanently outputted from pins 3-6 (Q0-Q3) to the Controller.
 - If the instruction requires an address (Operand), the **IO (Instruction Out)** signal is activated to push those bits onto the bus.
4. **Reset:** Activating the **Reset** line clears the current instruction to **0000 0000**

Function Table:

Reset	II (In)	IO (Out)	CLK	State/Operation
1	X	X	X	Reset: All internal bits become 0.
0	1	X	Trigger	Latched: IR holds previous data; ignores bus.
0	0	X	Trigger	Load: IR captures 8-bit data from the bus.
0	X	1	X	Output Disabled: Bus lines IR0-IR3 are Hi-Z.
0	X	0	X	Output Enabled: Operand bits move to bus.

Circuit Diagram:





Component List for IR Module:

Sl.	Component	Model	Quantity	Purpose
1	4-Bit D-Register	74LS173	2	Latches and stores the 8-bit instruction.
2	Hex Inverter	74HC04	2	Inverts control signals for proper triggering.
3	Logic Ports	Input/Output	16	To interface with the 8-bit System Bus.

CONTROL SEQUENCER

In this computer architecture, the control sequencer defines the microinstructions and provides control signal output for each machine cycle. The block diagram for it is given below,

Block diagram:

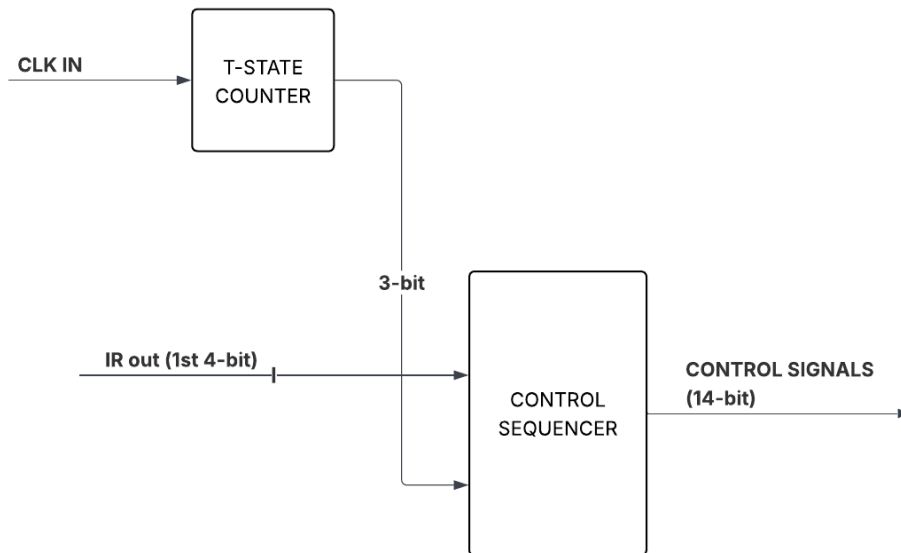


Fig: block diagram of control sequencer

T-States, Fetch, and Execution Cycles:

T-states (Timing states) represent individual clock cycles within a machine cycle. In the SAP-1 architecture, a single instruction is broken down into a sequence of these micro-steps to coordinate the movement of data between registers.

- **The T-State Counter:** A 3-bit counter (like the 74LS161) cycles through binary values 000 to 101 (T0 to T5). Each count triggers a specific "Control Word" from the EPROM.
- **The Fetch Cycle:** This occurs during **T0** and **T1**. It is universal for every instruction. The address is moved from the Program Counter to the Memory Address Register, the instruction is read from RAM, and it is loaded into the Instruction Register.
- **The Execution Cycle:** This occurs during **T2, T3 and T4**. The actions taken here depend entirely on the 4-bit opcode currently in the Instruction Register (e.g., LDA, ADD, SUB). During these states, the control unit toggles specific pins to perform the math or data transfers required by that specific command.

THE PROM:

[NOTE: For hardware implementation we would use the 28C16 EEPROM, as it is much easier to erase and reprogram. But this component is unavailable in proteus. So we instead used a 27C64 EPROM for software implementation.]

The **27C64** is an 8KB (64K-bit) UV-erasable Programmable Read-Only Memory. We are using the EPROM as a **Combinational Logic Replacement**. Instead of using dozens of logic gates to decode instructions, we use the EPROM's address pins as inputs and its data pins as outputs.

- **Inputs (Address Pins):** we connected the 4 bits from the Instruction Register and the 3 bits from the T-state counter to the address lines (A0–A6).
- **Outputs (Data Pins):** For every unique combination of "Instruction + Time," the EPROM outputs a 16-bit (using two chips in parallel) **Control Word**.

Control Pin Definitions

Based on your provided handwritten table, these pins represent the control signals that enable or disable specific parts of the computer:

- HLT: Stops the clock
- MI: MAR in
- RI: RAM in
- RO: RAM out
- IO: IR out
- II: IR in
- AI: Accumulator in
- AO: Accumulator out
- EO: sum out
- SU: Subtract
- BI: B REG in
- OI: Output REG in
- CE: Count enable
- CO: Counter out

The Binary File Structure:

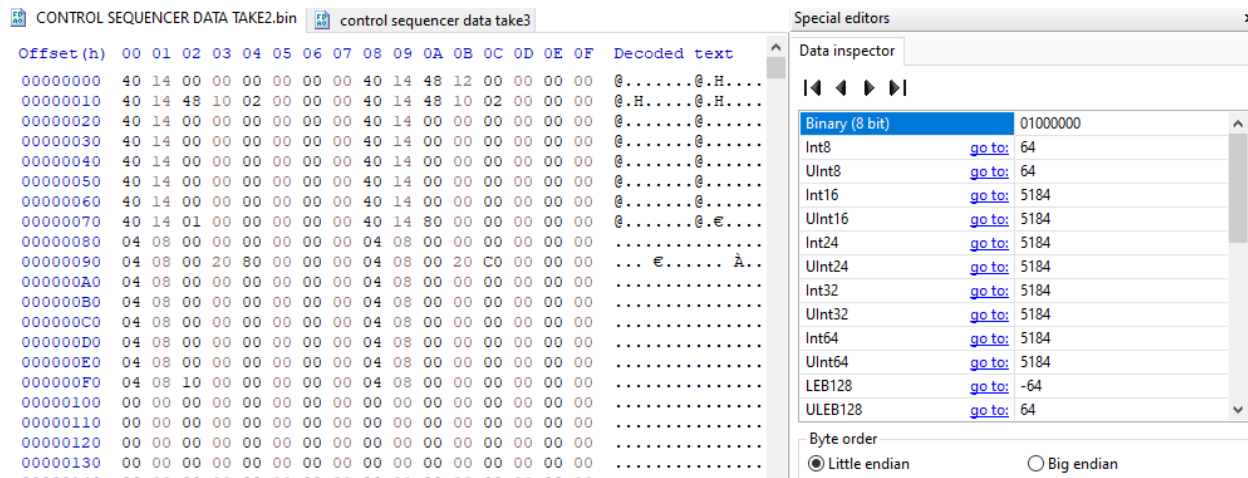


Fig: bin file written in HxD

A bin file is a raw data image. In the context of your 27C64 EPROM, it represents the exact state of every memory cell within the chip.

- **Address Mapping:** Each row in your HxD editor corresponds to a physical address in the EPROM. Since you are using the 4-bit Instruction Register and 3-bit T-state counter as inputs, these bits form the address.
- **Data Values:** The hex values you typed into HxD are the Machine Control Words. Each byte (or pair of bytes) represents the 1 or 0 state of the control pins (HLT, MI, RI, etc.) for that specific instruction and time step.

The Coding Process in HxD:

Creating this file involves translating your handwritten truth table into hexadecimal notation.

1. **Mapping Bits to Pins:** We assign each control signal to a specific bit in a byte. For instance, if MI is Bit 7 and RI is Bit 6, and we want only MI active, we would calculate the binary value 10000000.
2. **Padding:** Because a 27C64 has 8KB of space and your SAP-1 logic only uses a small fraction of that (the first 128 addresses if using 7 address bits), the rest of the bin file is usually filled with 00 to fill the remaining capacity of the chip.
3. **Firmware Generation:** Once saved, this .bin file acts as the virtual "firmware" that Proteus loads into the EPROM component.

CONTROL ROM TRUTH TABLE:

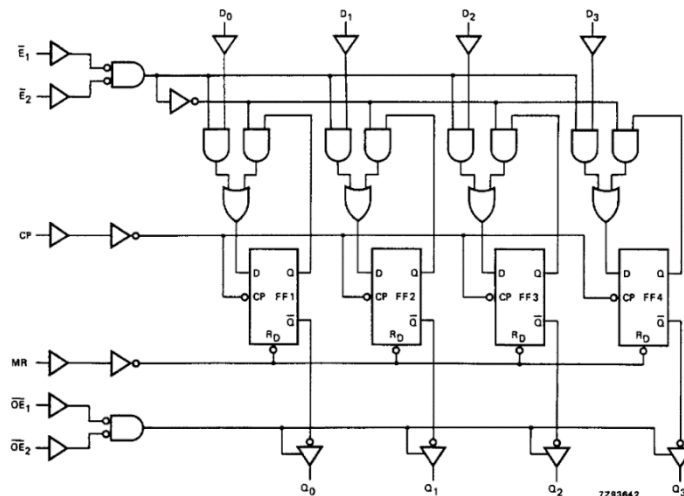
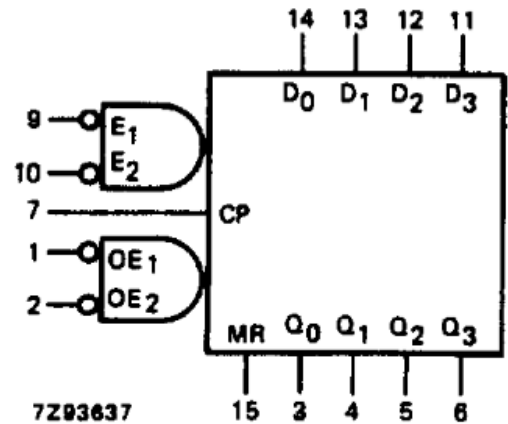
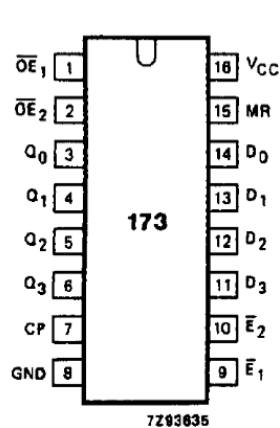
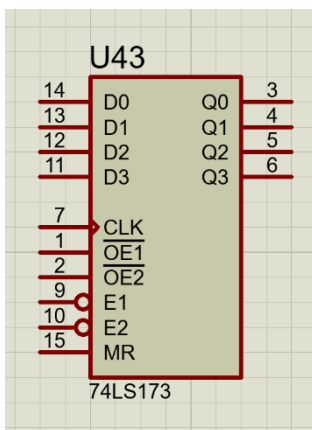
Instruction and fetch cycle	Opcode	STEP	HLT	MI	RI	RO	IO	II	AI	AO		EO	SU	BI	OI	CE	CO
fetch	XXXX	000	0	1	0	0	0	0	0	0		0	0	0	0	0	1
		001	0	0	0	1	0	1	0	0		0	0	0	0	1	0
LDA		010	0	1	0	0	1	0	0	0		0	0	0	0	0	0
		011	0	0	0	1	0	0	1	0		0	0	0	0	0	0
		100	0	0	0	0	0	0	0	0		0	0	0	0	0	0
ADD		010	0	1	0	0	1	0	0	0		0	0	0	0	0	0
		011	0	0	0	1	0	0	0	0		0	0	1	0	0	0
		100	0	0	0	0	0	0	1	0		1	0	0	0	0	0
SUB		010	0	1	0	0	1	0	0	0		0	0	0	0	0	0
		011	0	0	0	1	0	0	0	0		0	0	1	0	0	0
		100	0	0	0	0	0	0	1	0		1	1	0	0	0	0
OUT		010	0	0	0	0	0	0	0	1		0	0	0	1	0	0
		011	0	0	0	0	0	0	0	0		0	0	0	0	0	0
		100	0	0	0	0	0	0	0	0		0	0	0	0	0	0
HLT		010	1	0	0	0	0	0	0	0		0	0	0	0	0	0
		011	0	0	0	0	0	0	0	0		0	0	0	0	0	0
		100	0	0	0	0	0	0	0	0		0	0	0	0	0	0

FIG: CONTROL SEQUENCER DIAGRAM IN PROTEUS

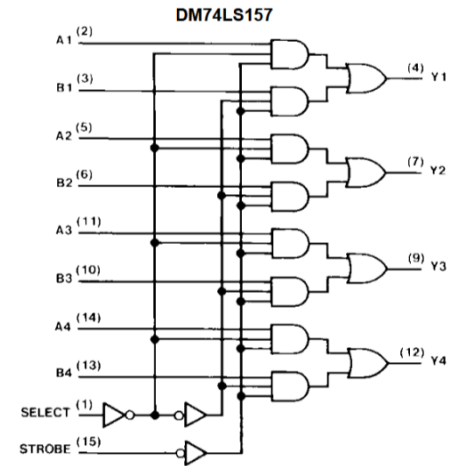
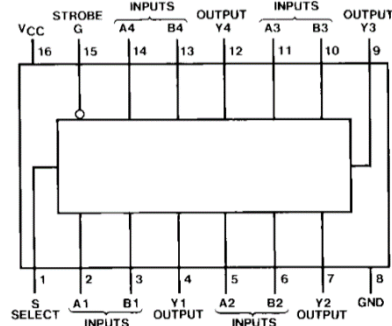
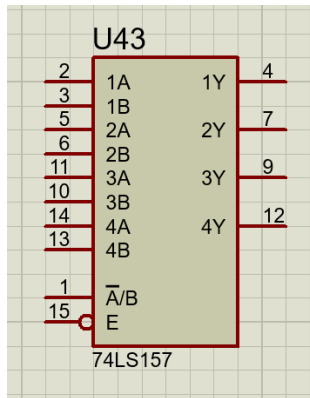
MAR & Address Selector

The Memory Address Selector, also known as MAR in short, in the SAP1 computer is a 4bit register that stores the memory address ready to get passed to RAM. On the other hand, Address Selector decides which address, either the one from the MAR or the one from Input Unit should be passed to RAM, depending on which mode the computer is in. It is generally a quad 2-to-1 MUX.

Introduction to 4bit D-type Flip Flop IC or 74LS173:



Introduction to Quad 2-to-1 MUX IC or 74LS157:



Function Table:

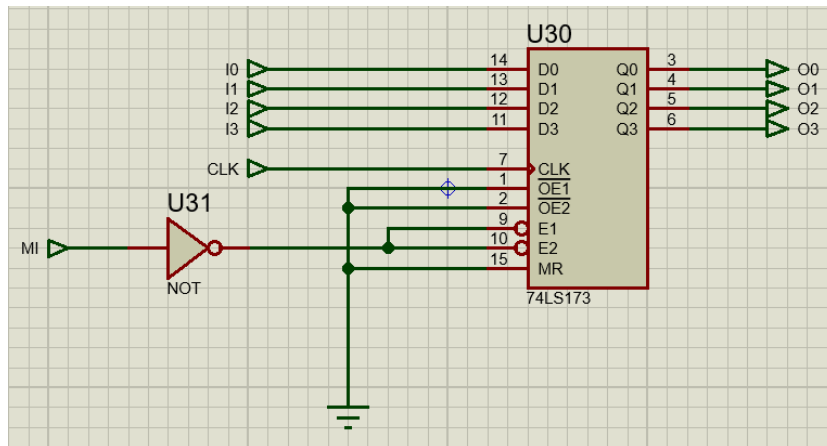
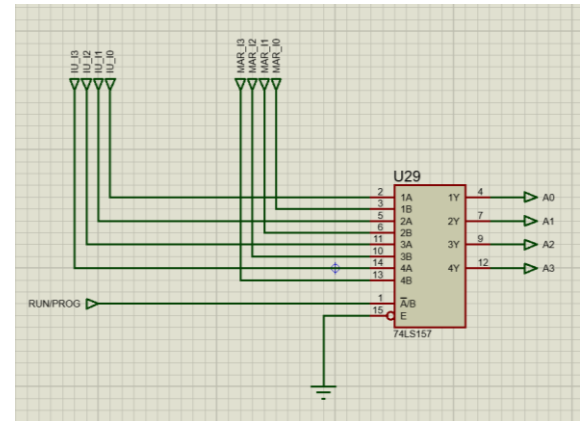
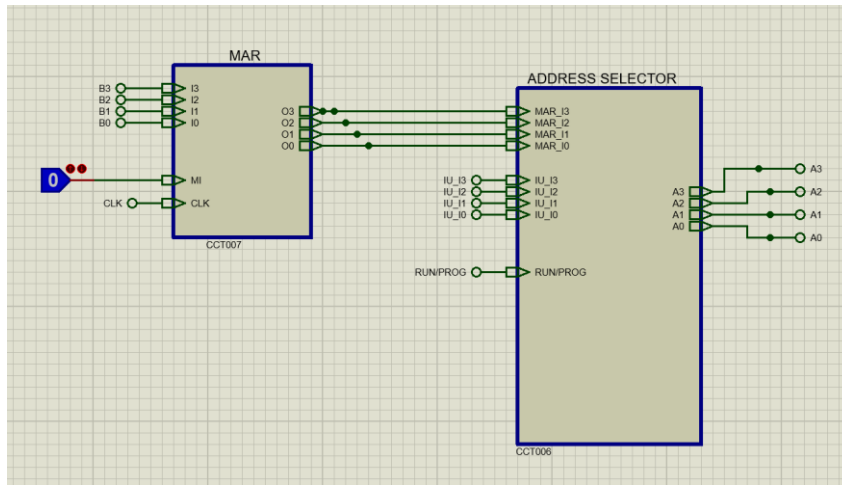
Fig. For 74LS173,

MR	CLK	E1'	E2'	OE1'	OE2'	Condition
1	X	X	X	X	X	No output
0	+ve edge	0	0	0	X	No output
				X	0	
		0	X	0	0	
		X	0			
		0	0	X	X	Data from bus gets stored
				0	0	Stored data shown at output

Fig. For 74LS157,

E	A'/B	Input		Output
		A	B	
1	X	X	X	0
0	0	Desired input		Shows A
	1			Shows B

Circuit Diagram:



Algorithm of MAR & Address Selector:

- When MI=0, no output is shown
- if MI=1 then data from bus is passed to Address Selector at each clock pulse.
- Address selector gets two 4bit address; one from MAR and other from Input Unit.
- When RUN/PROG=0, it is in Program mode which means the address from input unit will be passed to RAM.
- If RUN/PROG=1, it is in RUN mode which means address from MAR will be passed to RAM.

Component List for MAR & Address Selector:

Sl.	Component	Model	Quantity	Purpose
1	4-Bit D-Register	74LS173	1	Latches and stores the 4-bit address
2	Quad 2to1 MUX	74LS157	1	Decides which address is passed
3	Hex Inverter	74HC04	2	Inverts control signals for proper triggering.
4	Logic Ports	Input/Output	16	To interface with the 8-bit System Bus.

Discussion:

- **Limitation:**

1. **ALU:**

The primary **outcome** of the ALU is its ability to perform robust 8-bit addition and subtraction using 2's complement logic, providing accurate computational results to the system bus through a tri-state buffer. It successfully handles boundary cases, such as zero results and 8-bit overflows, ensuring data integrity for the Accumulator. However, its **limitations** include a lack of support for logical operations (AND, OR, NOT) and complex math like multiplication or division, which are not part of the basic SAP-1 architecture. Additionally, it lacks a "**Flags Register**," meaning the system cannot automatically detect carries or negative results for conditional branching

2. **Instruction Register:**

The **outcome** of the IR is the successful synchronization of the "**Fetch**" cycle, where it captures 8-bit instructions from the bus and splits them into a 4-bit Opcode for the controller and a 4-bit Operand for memory addressing. This modularity allows the computer to distinguish between "what to do" and "where to do it". The main **limitation** is its fixed 4-bit operand, which restricts the computer to only 16 bytes of total memory (**addresses 0000 to 1111**). Furthermore, as a volatile storage unit, it loses the current instruction immediately upon a reset or power loss.